TalentTrack, a Project Management System

Presented by:

Huy Gia Dinh
Kiera Pardo
Harvey Nghiem

1. Application Background

Companies often manage large amounts of information about employees, departments, work locations, supervisors, and internal projects. In many organizations, employees may be assigned to different projects, work under different supervisors, belong to specific departments, and operate from different company centers. As the size of a company grows, keeping track of these relationships becomes more difficult, especially when the information is spread across spreadsheets, emails, or separate internal systems.

Existing employee management platforms such as Workday, SAP SuccessFactors, BambooHR, and Oracle HCM provide large-scale human resources tools for managing employee records, payroll, benefits, hiring, onboarding, and compliance. These systems are powerful, but they are often designed as broad enterprise platforms that cover many HR functions at once. In contrast, our project focuses on a more specific use case: organizing employees around projects, supervisors, departments, and work centers. Rather than attempting to replace a full HR platform, our system is designed to demonstrate how a relational database can support project-based workforce tracking and structured querying.

Project-based work adds another layer of complexity. A single project may involve many employees from different departments, while one employee may contribute to multiple projects at the same time. Supervisors also need to oversee projects, monitor deadlines, and understand which employees are assigned to which responsibilities. Without a centralized database, it can be difficult for managers to answer basic questions such as which employees are working on a project, which supervisor is responsible for a project, how many employees work in each department or location, and whether a project has enough assigned workers.

Thus, this system is designed to help companies organize employee and project information in a structured relational database. The database keeps track of employees, departments, centers, supervisors, projects, tasks, salaries, and project assignments. It allows users to query important information such as employee counts by department or country, projects supervised by a specific manager, employees assigned to projects, and employees who have worked at the company for many years. By centralizing this information, the system supports better workforce organization, project tracking, and management decision-making.

2. Data Description

Our database has six main entities and several relationships. `Employee` stores information about each worker, including their employee ID, name, gender, start date, and annual salary. Each employee is part of one `Department`, such as Human Resources, Quality Assurance, or

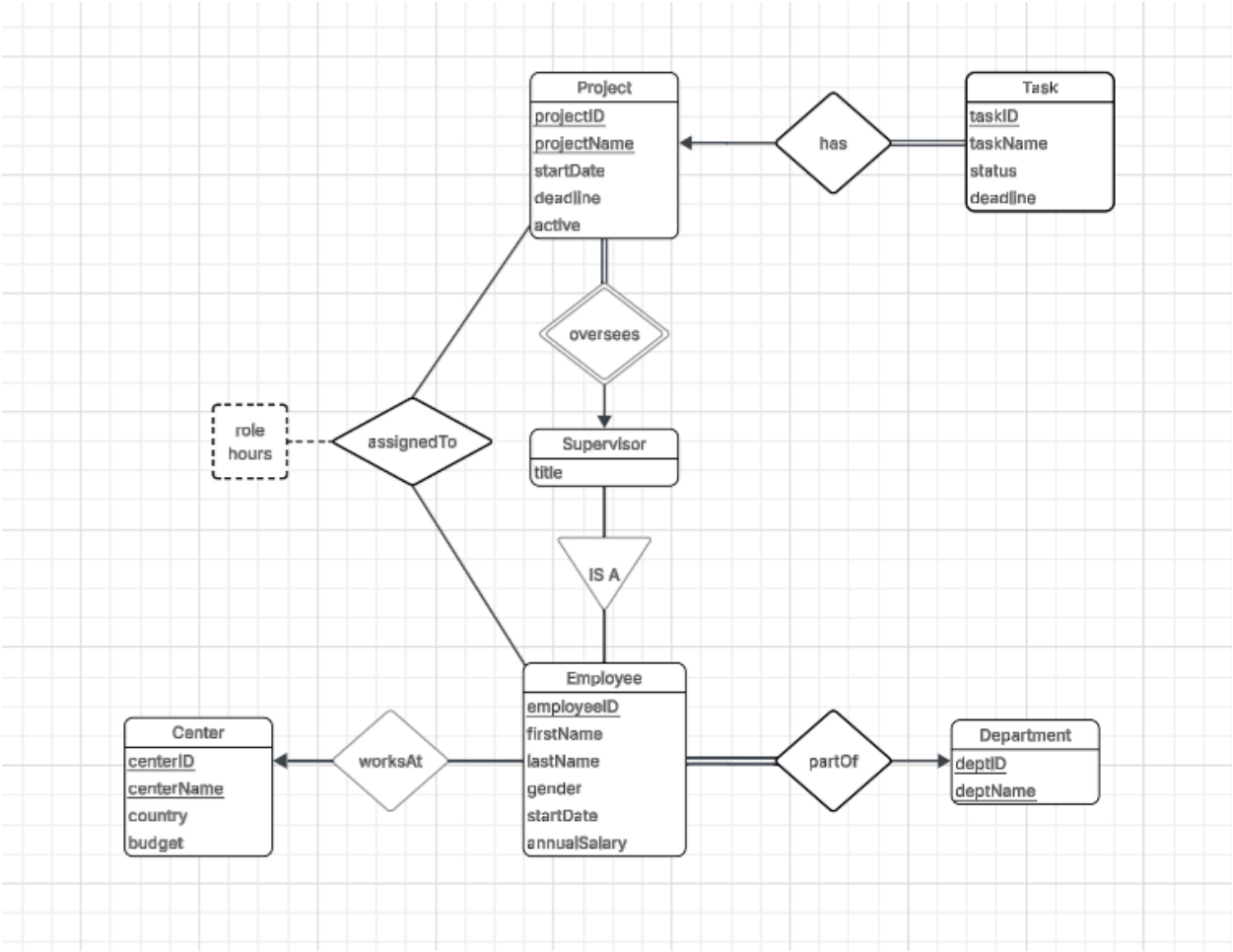
Engineering, and works at one `Center`, which stores information about the company location, country, and budget. Some employees are also Supervisors, which are modeled as a subtype of `Employee`. The `Supervisor` entity stores supervisor-specific information such as title, while still using the employee's general information from the `Employee` table.

The database also stores project-related information. `Project` stores each project's ID, name, start date, deadline, and whether the project is currently active. Each project is overseen by a supervisor, and employees may be assigned to one or more projects. The `AssignedTo` relationship records which employees are assigned to which projects, along with relationship-specific information such as the employee's role on the project and the number of assigned hours. This is necessary because a single employee can work on multiple projects, and a single project can have multiple employees. Finally, Task stores individual tasks connected to projects, including task names, statuses, and deadlines.

The main constraints in our database are primary key, foreign key, and NOT NULL constraints. Each entity has a primary key, such as employeeID, deptID, centerID, projectID, and taskID, which uniquely identifies each record. Foreign keys are used to preserve the relationships shown in the ER diagram. For example, `Employee` references `Department` and `Center`, `Supervisor` references `Employee`, `Project` references `Supervisor`, `Task` references `Project`, and `AssignedTo` references both `Employee` and `Project`. These constraints help ensure that the database does not contain invalid references, such as a project assigned to a non-existent employee or a task connected to a non-existent project.

The database is designed to reduce redundancy by separating repeated information into related tables. For example, department names are stored in the `Department` table instead of being repeated for every employee, and center information is stored in the `Center` table instead of being duplicated across employee records. The `AssignedTo` relationship also avoids storing multiple project columns inside the `Employee` table. Overall, the schemas are intended to satisfy at least Third Normal Form, and most relations satisfy BCNF because non-key attributes depend on the primary key rather than on other non-key attributes.

3. E/R Diagram



4. E/R Diagram to Relations

All major entities in the relational schema can be directly mapped from the ER diagram. Only a few changes were made to maintain normalization. The `Employee`, `Supervisor`, `Department`, `Center`, `Project`, and `Task` entities correspond directly to the diagram. All non-key attributes are dependent on their primary key in order to keep the schema in 3NF.

Looking at the `Employee` relation:

The `Employee` entity has foreign keys to both the `Department` and `Center`. There is a many-to-one relationship for the `Employee` and the `Department` and `Center` relations where many employees can work in a department, but an employee can only work under one department. The same relationship exists between the `Employee` and the `Center` relations.

The `Department` relation stores the department ID (`departmentID`) and department name (`departmentName`). `departmentID` is the primary key. The Center relation stores center ID (`centerID`), center Name (`centerName`), country, and budget. Since an employee can only work at one center, their residence can be determined by the country of the center

```
Employee(employeeID, firstName, lastName, gender,
startDate, annualSalary, deptID, centerID)
     FK: deptID references Department.deptID
         centerID references Center.centerID
Department(deptID, deptName)
Center(centerID, centerName, country, budget)
```

For the `Project` and `Task` relations, `Task` is a weak entity dependent on the `Project` entity. A task cannot exist without a project and a task can only fall under one project. A `Project` can only be overseen by one supervisor and must be supervised by a supervisor. A `Supervisor` can oversee many projects.

```
Supervisor(employeeID, title)
     FK: employeeID references Employee.employeeID

Project(projectID, projectName, startDate, deadline,
active, supervisorID)

Task(taskID,projectID, taskName, status, deadline)
     FK: projectID references Project.projectID
```

The `AssignedTo` relation has a many-to-many relationship, where an employee can be assigned to multiple projects and a project can be assigned to multiple employees. This is to reflect a realistic relationship between employees and projects, where employees don't solely work on just one project but multiple at a time.

```
AssignedTo(employeeID, projectID, role, hours)
     FK: employeeID references Employee.employeeID
     FK: projectID references Project.projectID
```

5. Functional Dependencies

These dependencies are based on the schemas in our database design. In general, each entity table uses a primary key to determine the rest of its attributes. The database is designed to avoid storing repeated information, such as department names inside the `Employee` table or project information inside the `AssignedTo` relationship.

Department

The `Department` table stores information about each department. Each department has a unique department ID, and each department ID determines the department name.

Functional dependency:

- `departmentID → {departmentID, departmentName}`

This is in BCNF because `departmentID` is a superkey.

If we also assume no two departments can have the same name, then:

- `departmentName → deptID, deptName`

In that case, `departmentName` would also be a candidate key.

Center

The `Center` table stores information about company work centers. Each center has a unique center ID, which determines the center name, country, and budget.

Functional dependency:

- `centerID → centerID, centerName, country, budget`

This is in BCNF because `centerID` is a superkey.

If we assume that each center name is unique, then:

- `centerName → centerID, centerName, country, budget`

However, if different countries can have centers with the same name, then `centerName` should not be treated as a key.

Employee

The Employee table stores general employee information, including name, gender, start date, salary, department, and center. These attributes are determined by the employee's unique employee ID.

Functional dependency:

- `employeeID → employeeID, firstName, lastName, gender, startDate, annualSalary, departmentID, centerID`

This relation is in BCNF because `employeeID` is a superkey.

Attributes such as `firstName` and `lastName` do not determine the employee because multiple employees could have the same name. Similarly, `departmentID` and `centerID` do not determine the employee because many employees can belong to the same department or work at the same center.

Supervisor

`Supervisor` is a subtype of `Employee`. Every supervisor is also an employee, so the `Supervisor` table uses `employeeID` as both its primary key and a foreign key referencing `Employee`. The supervisor's title is determined by the employee ID.

Functional dependency:

- `employeeID → employeeID, title`

This is in BCNF because `employeeID` is a superkey. The title alone does not determine the supervisor because multiple supervisors may have the same title.

Project

The `Project` table stores information about company projects. Each project has a unique project ID, which determines the project name, start date, deadline, active status, and supervisor.

Functional dependency:

- `projectID → projectID, projectName, startDate, deadline, active, supervisorID`

This relation is in BCNF because `projectID` is a superkey.

If we add the constraint that no two projects can have the same project name, then we also have:

- `projectName → projectID, projectName, startDate, deadline, active, supervisorID`

In that case, `projectName` would also be a candidate key. If project names are not guaranteed to be unique, then only `projectID` should be treated as the key.

Task

The `Task` table stores tasks associated with projects. Each task has a unique task ID, which determines the task name, status, deadline, and the project it belongs to.

Functional dependency:

- `taskID → taskID, taskName, status, deadline, projectID`

This relation is in BCNF because `taskID` is a superkey.

The `projectID` attribute does not determine a task because one project can have many tasks. Similarly, task names may repeat across different projects, so `taskName` should not be treated as a key unless the database explicitly enforces unique task names.

AssignedTo

`AssignedTo` represents the many-to-many relationship between `Employee` and `Project`. One employee can be assigned to multiple projects, and one project can have multiple employees assigned to it. The combination of `employeeID` and `projectID` determines the employee's role and assigned hours for that project.

Functional dependency:

- `{employeeID, projectID} → role, hours`

The full set of attributes determined by the key is:

- `{employeeID, projectID} → employeeID, projectID, role, hours`

This relation is in BCNF because `{employeeID, projectID}` is a superkey.

Neither `employeeID` alone nor `projectID` alone determines the remaining attributes. An employee may work on many projects, and a project may have many employees. Also, an employee's role may differ depending on the project, so role depends on the combination of employee and project rather than on the employee alone.

Overall Normalization

Overall, the relations are designed to be in BCNF, and therefore also satisfy 3NF, under the intended functional dependencies. Each table stores information about one entity or relationship, and non-key attributes depend on the primary key rather than on other non-key attributes.

For example, department names are stored in the `Department` table instead of being repeated inside `Employee`. Similarly, center information is stored in `Center` instead of being duplicated across employee records. The `AssignedTo` table also avoids storing multiple project columns inside `Employee` and correctly represents the many-to-many relationship between employees and projects. This design helps reduce redundancy and prevents update, insertion, and deletion anomalies.

6. Relational Schemas

Entities

```
CREATE TABLE Department (
    departmentID INT PRIMARY KEY,
    departmentName VARCHAR(100) NOT NULL
);
CREATE TABLE Center (
    centerID INT PRIMARY KEY,
    centerName VARCHAR(100) NOT NULL,
    country VARCHAR(100),
    budget DECIMAL(12,2)
);
CREATE TABLE Employee (
    employeeID INT PRIMARY KEY,
    firstName VARCHAR(100) NOT NULL,
    lastName VARCHAR(100) NOT NULL,
    gender VARCHAR(20),
    startDate DATE,
    annualSalary DECIMAL(12,2),
    centerID INT,
    departmentID INT,
    FOREIGN KEY (centerID) REFERENCES Center(centerID),
    FOREIGN KEY (departmentID) REFERENCES
Department(departmentID)
);
CREATE TABLE Supervisor (
    supervisorID INT PRIMARY KEY,
    title VARCHAR(100),
    FOREIGN KEY (supervisorID) REFERENCES Employee(employeeID)
);
CREATE TABLE Project (
    projectID INT PRIMARY KEY,
    projectName VARCHAR(150) NOT NULL,
    startDate DATE,
    deadline DATE,
    active BOOLEAN,
```

```
    supervisorID INT,
    FOREIGN KEY (supervisorID) REFERENCES
Supervisor(supervisorID)
);
CREATE TABLE Task (
    taskID INT,
    taskName VARCHAR(150) NOT NULL,
    status VARCHAR(50),
    deadline DATE,
    projectID INT,
    PRIMARY KEY (projectID, taskID),
    FOREIGN KEY (projectID) REFERENCES Project(projectID)
);
```

Relationship(s)

```
CREATE TABLE Task (
    taskID INT,
    taskName VARCHAR(150) NOT NULL,
    status VARCHAR(50),
    deadline DATE,
    projectID INT,
    PRIMARY KEY (projectID, taskID),
    FOREIGN KEY (projectID) REFERENCES Project(projectID)
);
```

7. Example Queries

The director of a department may want to see all employees working under their department. This query finds all supervisors working under the “Human Resources Department”:

```
SELECT e.*
FROM Employee e, Department d, Supervisor s
WHERE s.supervisorID = e.employeeID
     AND e.departmentID = d.departmentID
     AND d.departmentName = 'Human Resources Department';
```

Query Result:

employeeID	firstName	lastName	gender	startDate	annualSalary	centerID	departmentID
1117	Evelyn	Robinson	F	2018-05-17	178000.00	20	1
1554	Michael	Brown	M	2018-10-05	51000.00	52	1
1589	Aria	Evans	F	2016-12-12	125000.00	9	1
1628	Archer	King	M	2013-03-06	35000.00	26	1
1668	Benjamin	Harris	M	2019-07-09	117000.00	29	1
1712	Lucas	Lopez	M	2024-02-14	42000.00	44	1
1716	Henry	Green	M	2019-01-16	41000.00	1	1
1733	Zoe	King	F	2015-07-15	80000.00	26	1
1793	Isaac	Stewart	M	2022-12-05	67000.00	9	1
1853	Ava	Williams	F	2019-04-19	136000.00	25	1

In relational algebra the query that finds all supervisors working under the “Human Resources” is as follows:

```
π employeeID, firstName, lastName, gender, startDate,
annualSalary, departmentID, centerID
(
    σ departmentName = 'Human Resources'
    (
        Employee ⋈ Employee.departmentID =
Department.departmentID Department
    )
)
```

Mentioned previously, the country of residence is determined by the center the employee is working at. This is because realistically the employee would live in the same country that the center they work at is located. This query finds the number of employees located in “Germany”:

```
SELECT COUNT(*) AS employeeCount
FROM Employee e, Center c
WHERE e.centerID = c.centerID
     AND c.country = ‘Germany’;
```

Query Result:

country	employeeCount
Germany	110

In relational algebra the query that finds the number of employees working in “Lebanon” is as follows:

```
γ COUNT(e.employeeID) → employeeCount (
     σ c.country = 'Germany' (
          Employee e ⋈ e.centerID = c.centerID Center c
     )
)
```

A director may want to see what projects a specific supervisor is assigned to, along with the associated deadlines with their projects. This query returns all project names and deadlines that a supervisor named “Oliver Bell” oversees:

```
SELECT p.projectName, p.deadline
FROM Project p, Employee e
WHERE p.supervisorID = e.employeeID
     AND e.firstName = 'Oliver'
     AND e.lastName = 'Bell';
```

Query Result:

projectName	deadline
Obsidian Crypto Custody - Modernization	2022-01-21
Vertex Market Data - Security Hardening	2021-02-22
Nexus Trading - Sunset / Replace	2023-07-09
Copperfield Derivatives Compliance Refresh	2021-05-12

In relational algebra the query that returns all project names and deadlines that supervisor “Oliver Bell” oversees is as follows:

```
π projectName, deadline
(
 σ firstName = 'Oliver' ∧ lastName = 'Bell'
     (
                Project ⋈ Project.supervisorID =
Employee.employeeID Employee
     )
)
```

When making promotions, it may be helpful to find employees by their years of experience. The following query returns all employees that have worked at the company for 20 years, return their name, salary, and department.

```
SELECT e.firstName, e.lastName, e.annualSalary,
d.departmentName
```

```
FROM Employee e
JOIN Department d ON e.departmentID = d.departmentID
WHERE TIMESTAMPDIFF(YEAR, e.startDate, CURDATE()) >= 20;
```

Query Result:

firstName	lastName	annualSalary	departmentName
Alexander	Harris	155000.00	Finance and Accounting
Ashley	Edwards	180000.00	Finance and Accounting
James	Taylor	175000.00	Finance and Accounting
Carter	Hunter	185000.00	Finance and Accounting
Amanda	Scott	72000.00	Finance and Accounting
Joshua	Thompson	196000.00	Finance and Accounting
Grace	Clark	150000.00	Finance and Accounting
Henry	Anderson	99000.00	Finance and Accounting
David	Scott	91000.00	Finance and Accounting
Ethan	Davis	197000.00	Finance and Accounting
James	Jackson	51000.00	Finance and Accounting
Alexander	Hall	129000.00	Information Technology
James	Miller	118000.00	Information Technology
Evelyn	Jackson	138000.00	Information Technology
Noah	Martinez	91000.00	Information Technology
Alexander	Harris	114000.00	Information Technology
Benjamin	Adams	43000.00	Information Technology
Steven	Stewart	115000.00	Information Technology
Linda	James	67000.00	Information Technology
Tyler	Carter	185000.00	Information Technology
Oliver	Johnson	70000.00	Information Technology
Jackson	Martinez	163000.00	Information Technology
Isabella	Davis	121000.00	Information Technology
Scarlett	Allen	76000.00	Information Technology
Samuel	Adams	108000.00	Information Technology
Henry	Lewis	43000.00	Engineering
George	Roberts	166000.00	Engineering
Susan	Edwards	155000.00	Engineering
Nicole	Hernandez	163000.00	Engineering
Noah	Davis	83000.00	Engineering
Emily	Lee	81000.00	Engineering
Sarah	Adams	68000.00	Engineering
Aria	Perez	168000.00	Engineering
Kathleen	Phillips	111000.00	Engineering
Laura	Morris	187000.00	Engineering
Aria	Bailey	158000.00	Sales and Marketing
Peyton	Price	127000.00	Sales and Marketing
Andrew	Hall	101000.00	Sales and Marketing
Alexis	Stewart	67000.00	Sales and Marketing
Quinn	Reed	150000.00	Sales and Marketing
Jessica	Thomas	85000.00	Sales and Marketing
Patricia	Lee	145000.00	Sales and Marketing
Mia	Bennett	168000.00	Sales and Marketing

In relational algebra the query that returns all employees that have worked at the company for 20 years, return their name, salary, and department is as follows:

```
π firstName, lastName, annualSalary, deptName (
    σ TIMESTAMPDIFF(YEAR, startDate, CURRENT_DATE) >= 20 (
        Employee ⋈ Employee.deptID = Department.deptID
Department
    )
)
```

Finally, when assigning projects to supervisors it would be helpful for a director to see which supervisors are not overseeing any projects so that they may assign them a project. The following query returns all supervisors who are not overseeing any projects:

```
SELECT e.employeeID, e.firstName, e.lastName
FROM Supervisor s, Employee e
WHERE s.employeeID = e.employeeID
```

```
        AND NOT EXISTS (
        SELECT *
        FROM Project p
        WHERE p.supervisorID = s.employeeID
             AND p.active = TRUE
    );
```

Query Result:

employeeID	firstName	lastName
1032	Zoe	Scott
1076	Elijah	Roberts
1083	Hannah	Stewart
1105	Olivia	Wilson
1117	Evelyn	Robinson
1121	Sofia	Lee
1141	Jonathan	Collins
1154	Sophia	Lee
1180	Nora	Stewart
1184	Chloe	Price
1195	Carter	Hunter
1197	Christopher	Stevens
1213	Joseph	Thomas
1239	Andrew	Hall
1246	Amanda	Scott
1251	Emma	Johnson
1266	Henry	Jackson
1286	Samuel	Adams
1294	Grayson	Campbell
1298	Eli	Collins
1329	Zoe	Nelson
1345	Asher	Price
1367	Grace	Moore
1372	Jackson	Nguyen
1391	Savannah	Hill
1401	Emma	Smith
1437	Audrey	Green
1444	Lincoln	Richardson
1452	Michael	Brown
1458	Brian	Hernandez
1540	Jackson	Ramirez
1554	Michael	Brown
1578	Oliver	Scott
1607	Sophia	Hernandez

In relational algebra the query that returns all supervisors who are not overseeing any projects is as follows:

```
    π employeeID, firstName, lastName
        (
             (Supervisor ⋈ Supervisor.employeeID =
        Employee.employeeID Employee)
        -
        π employeeID, firstName, lastName
        (
             (Supervisor ⋈ Supervisor.employeeID =
        Employee.employeeID Employee)
             ⋈ Supervisor.employeeID = Project.supervisorID
             (σ active = TRUE (Project))
        )
```

)

8. Implementation: Technologies Used

We used MySQL as our DBMS for the backend of the application. MySQL allowed us to create and manage the relational database tables, enforce primary key and foreign key constraints, and test SQL queries involving joins, aggregation, grouping, and subqueries. We also used MySQL Workbench to create the database, import data, view table contents, and test our SQL queries before connecting them to the application.

For the command-line interface, we used Java with JDBC. JDBC allows the Java program to connect directly to the MySQL database, execute SQL queries, and display the results in the terminal. Our CLI provides menu-based options so users can run predefined queries, choose specific fields, such as different departments or countries, etc. Furthermore, it also allows users to type their own SQL statements if they wish to do so. The Java program connects to the local MySQL server using the database URL, username, and password, then sends SQL commands through JDBC.

For query execution, we stored our main SQL queries inside the Java CLI as predefined menu options. When a user selects an option, the CLI runs the corresponding SQL query and prints the results in a formatted table. In addition to fixed queries, we added interactive search features that allow users to search employees by different fields, such as department, center, country, and other employee-related attributes. This makes the CLI more flexible because users are not limited to only one preset query result; they can provide input and retrieve more specific information from the database. The MySQL server and Java CLI are currently run locally. We used Git/GitHub for version control and collaboration, allowing group members to share SQL scripts, Java code, and project documentation.

9. WebApp Code/Screenshots

Link to Github: https://github.com/DinhHuyGia/CSDS-341-Final-Project

10. Role of Members

Huy

- Created database tables using SQL
- Built the CLI using Java and connected it with the MySQL database with JDBC
- Determined functional dependencies and wrote section 1, 2, 5, and 8.
- Updated the CLI to make it more interactive, allowing users to enter their own queries or search with their own values.

Kiera

- Made the ER diagram.
- Made the SQL queries to be used for the CLI as preexisting options for the user to use.
- Determined functional dependencies and wrote section 3, 4, and 7.

Harvey

- Created the relational schema.
- Generated missing data using Python and cleaned the data.
- Helped with writing java interface and integration.
- Wrote section 6 and 9.

11. What We Learned

Huy

Through this project, I learned how to design a relational database from an ER diagram and converted entities, relationships, and constraints into SQL schemas. Furthermore, I gained a better understanding of normalization, especially how separating repeated information into separate tables prevents update problems. On the implementation side, I learned how to wire our Java CLI to the database on MySQL using JDBC. This helped me better understand how applications communicate with databases outside of MySQL Workbench.

Kiera

I learned how to write a database system and think critically about what entities and relationships would need to exist through creating the ER diagram. It helped me to better understand decision making when it came to designing a database and how to avoid redundancies. Additionally, I

improved on writing in SQL, a language I never used before to create queries that a user could use. Working on representing the queries as relational algebra, I was also able to improve my understanding of relational algebra and to better understand the logic behind each query.

Harvey

From this project, I have learned how to design a database from the ground up, from underlying the problem to understanding the objective and designing a suitable database that can handle these queries. I also learnt how to clean and format the dataset to be suitable for the database. This project has also teached me the basics of databases in industry. Working in a group has helped me improve my understanding of database management and improve my soft skills.